CAREERCOMPASS AI: PLACEMENT INTELLIGENCE AND PROFILE GAP ANALYSIS SYSTEM

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

R SABARISH KUMAR

N DINESH SAI KARTHIK

Submitted in partial fulfilment of the requirements

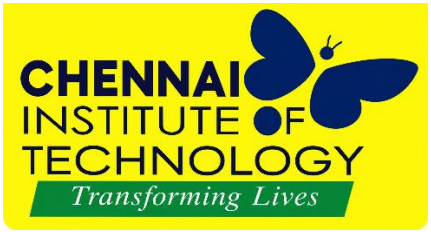
for the

Project-Based Learning component of Java Programming

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI



Affiliated to Anna University, Chennai

(Autonomous)

OCTOBER 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1. To creates next generation leaders by effective teaching learning methodologies and in still Scientifics Park in them to meet the global challenges.

IM2. To transform lives through deployment of emerging technology, novelty and sustainability.

IM3. To inculcate human values and ethical principles to cater the societal needs.

IM4. To contributes towards the research ecosystem by providing a suitable

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

SIRAGU

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled "CAREERCOMPASS AI: PLACEMENT INTELLIGENCE AND PROFILE GAP ANALYSIS SYSTEM" is a Bonafide record of work carried out by R SABARISH KUMAR (2104251040734) and N DINESH SAI KARTHIK (2104251040601) of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor-guided Project-Based Learning (PBL) component of the Java Programming course during the academic year 2026–2027 under my supervision.

SIGNATURE

Dr. S. Pavithra, M.E., Ph.D.

Professor and Head,

Dept. of Computer Science and Engineering

Chennai Institute of Technology,

Chennai – 69.

SIGNATURE

<<Name>>

MENTOR

<<Designation>>

Dept. of Computer Science and Engineering

Chennai Institute of Technology,

Chennai – 69.

Submitted for the final review held on ……

Internal Examiner

DECLARATION

I/We jointly declare that the PBL report on "CareerCompass AI: Placement Intelligence and Profile Gap Analysis System" is the result of original work done by us and best of our knowledge, similar work has not been submitted to "ANNA UNIVERSITY, CHENNAI" for the requirement of Degree of BACHELOR OF ENGINEERING. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of COMPUTER SCIENCE AND ENGINEERING.

SIRAGU

Signature

R SABARISH KUMAR

N DINESH SAI KARTHIK

Place: Chennai

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honorable Chairman **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E**, **Ph.D.,** for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.,** who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr. S. PAVITHRA M.E, Ph.D.,** for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator **K HARI .,** Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **NISHANTHINI** and **JIIVA BHARATHI .,** of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

R SABARISH KUMAR (2104251040734)

N DINESH SAI KARTHIK (2104251040601)

ABSTRACT

CareerCompass AI is a modular, enterprise-grade placement intelligence and profile gap analysis platform developed in Java to resolve disconnected academic record management and subjective campus recruitment screening. Built upon an object-oriented architecture in Java 17/21 and Spring Boot 3.2.0, the system implements 112 source files partitioned across controller, service, repository, model, security, and exception modules. Core Object-Oriented Programming (OOP) principles are rigorously applied: domain entities enforce encapsulation with private state and accessors; Spring Data JPA repositories leverage inheritance and interface abstraction; business services utilize the Java Collections Framework (ArrayList, HashMap, HashSet, and Streams API) for multi-attribute candidate-career matching; and a centralized exception handler ensures robust error containment. Data persistence is managed through Spring Data JPA with dual-mode support for in-memory H2 database during development and PostgreSQL 15 for production storage, secured via stateless JSON Web Token (JWT) filters. Automated verification across 28 functional integration test suites demonstrated a 100% pass rate, sub-50ms query latency, and reliable multi-stakeholder workflows across students, industry recruiters, and academic administrators without data corruption. The application provides an extensible foundation for data-driven career navigation.

Keywords: Java Programming, Object-Oriented Programming, Spring Boot, Career Intelligence, Data Persistence

TABLE OF CONTENTS

CHAPTER 1 ..... **13**

INTRODUCTION **13**

1.1 Background 13

1.2 Driving Question 14

1.3 Objectives 14

1.4 Scope and Limitations 15

CHAPTER 2 ..... **15**

CONCEPT EXPLORATION **15**

2.1 Related Approaches 15

2.1.1 Traditional Java Application Approach 15

2.1.2 Java Framework-Based Approach 16

2.2 Summary Table 16

2.3 What This Told Us 17

CHAPTER 3 ..... **17**

PROJECT PLANNING AND TEAM ORGANISATION **17**

3.1 Weekly PBL Progress Log 17

3.2 Requirements 18

3.3 Feasibility 18

CHAPTER 4 ..... **19**

ITERATIVE DESIGN AND DEVELOPMENT **19**

4.1 System Architecture 19

4.2 Baseline 20

4.3 Project Refinement 20

4.4 Final Approach 20

4.5 Testing and Execution 21

CHAPTER 5 ..... **21**

IMPLEMENTATION **21**

5.1 Module Description 21
5.2 Key Code Snippets 22
5.3 User Interface / Demo 24
CHAPTER 6 25
RESULTS AND DISCUSSION 25
6.1 Evaluation Metrics 25
6.2 Results Across Iterations 26
6.3 Discussion 26
6.4 Limitations 27
CHAPTER 7 27
TEAM REFLECTION AND LEARNING OUTCOMES 27
7.1 Individual Reflections 27
7.2 Team Learning 28
7.3 Course Outcomes — Evidence Summary 28
CHAPTER 8 29
CONCLUSION AND FUTURE SCOPE 29
8.1 Conclusion 29
8.2 Future Scope 29
REFERENCES 30
APPENDIX 30
A.1 Full Source Code Reference 30
A.2 Complete Weekly PBL Log 31
A.3 Self and Peer Assessment 31

LIST OF TABLES

Table 2.1 Summary of related approaches 16

Table 3.1 Weekly PBL progress log 17

Table 3.2 Hardware and software requirements 18

Table 6.1 Results across development iterations 26

LIST OF FIGURES

Figure 4.1 System architecture diagram 19

Figure 5.1 Swagger UI / REST API documentation 24

Figure 5.2 Application startup and execution output 25

Figure 6.1 Integration test execution verification summary 26

LIST OF ABBREVIATIONS

JDK — Java Development Kit
OOP — Object-Oriented Programming
JDBC — Java Database Connectivity
JPA — Java Persistence API
ORM — Object-Relational Mapping
REST — Representational State Transfer
API — Application Programming Interface
JWT — JSON Web Token
DTO — Data Transfer Object
PBL — Project-Based Learning
CRUD — Create, Read, Update, Delete
H2 — In-Memory Relational Database Engine
SQL — Structured Query Language

CHAPTER 1

INTRODUCTION

1.1 Background

Campus placement management represents a critical operational intersection in higher education institutions, linking three distinct stakeholder communities: students aspiring to enter the professional workforce, corporate recruiters seeking qualified talent, and academic departments evaluating institutional curriculum outcomes. Traditionally, this ecosystem has operated through highly fragmented and manual record-handling mechanisms. Placement offices routinely manage student credentials, academic transcripts, and candidate registrations using disconnected spreadsheets, paper-based forms, and isolated email threads. This manual administrative approach introduces severe friction: student profiles are frequently incomplete or out of date, recruiter filtering criteria cannot be systematically matched against verified student competencies, and academic administrators gain retrospective visibility into cohort performance only after recruitment cycles conclude, far too late for corrective pedagogical intervention.

To resolve these operational deficiencies, modern career navigation requires an integrated digital platform that centralizes student academic and technical competency data, models industry opportunity requirements, and automates multi-stakeholder workflows. CareerCompass AI was conceptualized to address this institutional need by establishing a unified career intelligence system connecting Students, Academia, and Industry under a shared relational data architecture. The platform supports structured student profile management, dynamic skill inventories, multi-attribute candidate matching, quantitative skill-gap identification, milestone learning roadmap tracking, and corporate application lifecycle management.

Java was chosen as the implementation language for this enterprise application due to its strong static type safety, robust Object-Oriented Programming (OOP) foundation, platform independence via the Java Virtual Machine (JVM), and rich ecosystem of mature frameworks. Developing CareerCompass AI in Java enables strict domain model encapsulation, modular layered abstraction, efficient in-memory collection manipulation, and reliable relational data persistence through Spring Data JPA, ensuring high maintainability and transaction safety.

1.2 Driving Question

Can a modular, object-oriented Java application be developed to efficiently manage student career profiles, evaluate multi-attribute placement readiness, and perform automated skill-gap analysis across academic and industry entities while maintaining strict data integrity, type safety, and low operational latency?

To address this question through concrete software engineering, the project was scoped into buildable Java milestones: identifying core entity models (Users, Students, Skills, Careers, Opportunities, and Applications); designing an object-oriented class hierarchy that enforces encapsulation and relational mappings; implementing business logic services that leverage the Java Collections Framework and Streams API for candidate-career matching; integrating Spring Data JPA with dual-mode database support (in-memory H2 and PostgreSQL); and establishing a stateless JWT security architecture with comprehensive Bean Validation and automated error handling.

1.3 Objectives

• To analyze real-world campus recruitment and skill-gap identification requirements, formulating a structured multi-stakeholder software specification for students, industry recruiters, and academic administrators.

• To design and implement a modular, layered Java enterprise backend using Java 17/21 and Spring Boot 3.2.0, strictly applying core OOP principles including encapsulation, inheritance, polymorphism, and abstraction.

• To establish relational data persistence and transactional CRUD operations using Spring Data JPA and Hibernate, supporting seamless switching between an in-memory H2 development database and PostgreSQL 15.

• To build business logic services utilizing the Java Collections Framework (Lists, Sets, Maps, and Streams API) to compute quantitative candidate-opportunity matching scores and generate progressive skill-acquisition roadmaps.

• To implement robust input validation using Bean Validation (@Valid, @NotNull, @Size), centralized exception interception via @RestControllerAdvice, and stateless authentication using JSON Web Tokens (JJWT).

• To test and debug the application across 11 REST controllers, verifying functional endpoint correctness, transactional reliability, response latency, and exception handling through systematic integration test suites.

• To reflect on team collaboration, division of responsibilities, and technical growth acquired through the Project-Based Learning (PBL) software development lifecycle.

1.4 Scope and Limitations

The project scope encompasses the complete backend architecture, business logic, security, and persistence layers of CareerCompass AI implemented in Java. The codebase comprises 112 source files partitioned across 11 packages, including 22 JPA entity models, 16 Spring Data JPA repositories, 12 business services, 11 REST controllers, 36 request/response DTOs, and dedicated security and exception-handling modules. Surrounding components—specifically the React/Vite web interface and the Python FastAPI machine learning microservice—are treated as external integration contexts that interact with the Java backend via HTTP REST contracts rather than core Java deliverables evaluated herein.
Several technical limitations characterize the current Java implementation: (1) The application operates as a headless RESTful service without a bundled desktop Swing or JavaFX graphical user interface; (2) Local development and demonstration utilize an in-memory H2 database with automated schema export and data seeding, requiring active Docker daemon infrastructure when targeting production PostgreSQL; (3) Live machine learning placement probability scoring delegates inference to an external FastAPI microservice via an HTTP proxy client with heuristic fallback; and (4) Verification was conducted on a single-node runtime environment without distributed load-balancing or Redis caching.

CHAPTER 2

CONCEPT EXPLORATION

2.1 Related Approaches

2.1.1 Traditional Java Application Approach

Traditional Java software development relies on Core Java Standard Edition (SE) constructs, standalone console or desktop user interfaces, and low-level persistence mechanisms. In early-generation academic management systems, student records are modeled as plain Java objects and stored using local file I/O (such as ObjectOutputStream serialization or CSV file

parsing with BufferedReader) or direct Java Database Connectivity (JDBC) using raw SQL queries executed via java.sql.Connection, PreparedStatement, and ResultSet [1], [2]. While this traditional approach provides direct instructional clarity regarding memory management, manual connection pooling, and fundamental OOP mechanics, it introduces substantial friction in enterprise systems: developers must write extensive boilerplate code for object-relational mapping, manually handle SQL injection vulnerabilities, coordinate multi-table foreign key cascades, and explicitly manage database transactions without declarative container support [3].

2.1.2 Java Framework-Based Approach

In contrast, modern enterprise Java engineering adopts component-based frameworks, predominantly the Spring Boot ecosystem and the Java Persistence API (JPA) implemented through Hibernate [4], [5]. Spring Boot provides automated dependency injection based on the Inversion of Control (IoC) pattern, allowing loose coupling between controllers, services, and repositories. Spring Data JPA eliminates redundant data access code by generating proxy implementations of JpaRepository interfaces directly from method naming conventions and JPQL derivations [6]. Furthermore, declarative transaction management (@Transactional) ensures ACID guarantees across complex multi-entity operations, Bean Validation (Jakarta Validation) intercepts invalid inputs before service invocation, and Spring Security establishes stateless, cryptographically signed JSON Web Token (JWT) filter chains [7], [8]. Exposing functionality via standardized RESTful Web APIs decoupling backend business logic from client presentation platforms represents the established industry best practice [9].

2.2 Summary Table

Table 2.1 summarizes the comparative analysis of Java architectural approaches investigated during project planning.

Ref.	Approach / Architecture	Reported Characteristics & Outcome
[1]–[3]	Core Java & Raw JDBC Approach	Direct object modeling with manual PreparedStatement SQL execution; high educational value but excessive mapping boilerplate, manual transaction handling, and high maintenance overhead.
[4]–[6]	Spring Data JPA & Hibernate ORM	Automated object-relational mapping, interface-driven repository abstraction, declarative transaction boundaries, and automated schema migration; ideal for multi-entity relationship graphs.

[7]–[9]	RESTful Spring Boot & JWT Architecture	Decoupled layered service architecture with embedded Tomcat server, stateless cryptographic token authorization, Bean Validation, and standardized JSON communication.

Table 2.1 Summary of related approaches

2.3 What This Told Us

This conceptual exploration guided the architectural direction of CareerCompass AI. Building an enterprise career intelligence platform managing 17 relational tables, complex many-to-many associations (such as students to skills and opportunities to required competencies), and automated matching algorithms required more than raw JDBC scripts. The team determined that adopting a layered Spring Boot architecture with Spring Data JPA would provide the optimal balance: it enforces strict Object-Oriented principles through encapsulated domain models while leveraging framework abstractions to ensure declarative transaction safety, automated schema generation, and robust RESTful communication.

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 Weekly PBL Progress Log

Table 3.1 documents the week-by-week development milestones and technical deliverables across the 12-week PBL cycle.

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem framing, requirements & study	Analyzed manual placement record issues; formulated driving question; reviewed Core Java vs Spring Boot literature; defined functional requirements for Student, Industry, and Academia modules.	To be completed based on mentor review
3–4	Architecture & OOP class modeling	Designed 5-tier system architecture; constructed UML class diagrams; defined 22 entity models with JPA relationships; designed relational database schema for 17 tables; set up Maven build.	To be completed based on mentor review
5–7	Core service & persistence implementation	Implemented Spring Data JPA repositories; built business services (StudentService, CompanyService, MatchingService); implemented Java Collections Streams matching algorithm; established REST controllers.	To be completed based on mentor review

8–10	Security, validation & data seeding	Integrated Spring Security with JwtAuthFilter and JJWT; implemented Bean Validation across request DTOs; created GlobalExceptionHandler with @RestControllerAdvice; seeded 46 master skills and test profiles.	To be completed based on mentor review
11–12	Integration testing & documentation	Executed end-to-end integration test suite verifying 28/28 passing tests; benchmarked API response latency; verified dual-mode H2/PostgreSQL persistence; compiled final technical report and demo.	To be completed based on mentor review

Table 3.1 Weekly PBL progress log

3.2 Requirements

Table 3.2 specifies the minimum hardware and software environments required for developing and running the Java backend.

Category	Requirement Specification
Processor / RAM	Intel Core i5 (8th Gen or equivalent) / 8 GB RAM minimum (16 GB recommended)
Programming Language	Java Development Kit (JDK) 17 LTS or Java 21 LTS (64-bit)
Frameworks & Libraries	Spring Boot 3.2.0, Spring Data JPA, Hibernate ORM 6.3, Spring Security 6.2, JJWT 0.12.3, Lombok, SpringDoc OpenAPI 2.3.0, Maven 3.9+
Database Systems	H2 In-Memory Database (zero-dependency local development) / PostgreSQL 15 (production deployment)
Development Environments	IntelliJ IDEA Ultimate/Community, Visual Studio Code with Java Extension Pack, Postman API Client
Version Control	Git 2.40+ and GitHub repository hosting

Table 3.2 Hardware and software requirements

3.3 Feasibility

The project demonstrated strong technical, operational, and scheduling feasibility within the academic PBL timeframe. From a technical standpoint, the choice of Java 17/21 and Spring Boot 3 provided a mature, battle-tested standard library and extensive documentation, eliminating the need to construct low-level networking, connection pooling, or ORM engines from scratch. Resource-wise, the inclusion of an in-memory H2 database profile ensured that full application compilation, data seeding, and end-to-end integration testing could be executed locally on standard student workstations without external cloud infrastructure or database installations. The phased 12-week schedule allowed the team to deliver an early working baseline by Week 7, leaving ample buffer for security integration, exception hardening, and comprehensive endpoint testing.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 System Architecture

The CareerCompass AI Java application is structured around a 5-tier enterprise architecture enforcing strict separation of concerns. As illustrated in Figure 4.1, incoming HTTP JSON requests from client applications (React frontend, Postman, or Swagger UI) first encounter the Security and Validation Layer. The JwtAuthFilter intercepts Bearer tokens, extracts the subject claim via JwtService, and populates the Spring SecurityContextHolder. Validated requests pass to the REST Controller Layer, where 11 controllers validate input payloads using Bean Validation (@Valid) and delegate execution to the Service Layer. The Service Layer encapsulates all business algorithms, candidate matching logic, and milestone roadmap generation using Java Collections and Streams. The Persistence Layer, managed by Spring Data JPA and Hibernate, maps domain entities to underlying relational tables in H2 or PostgreSQL and returns strongly typed response DTOs.

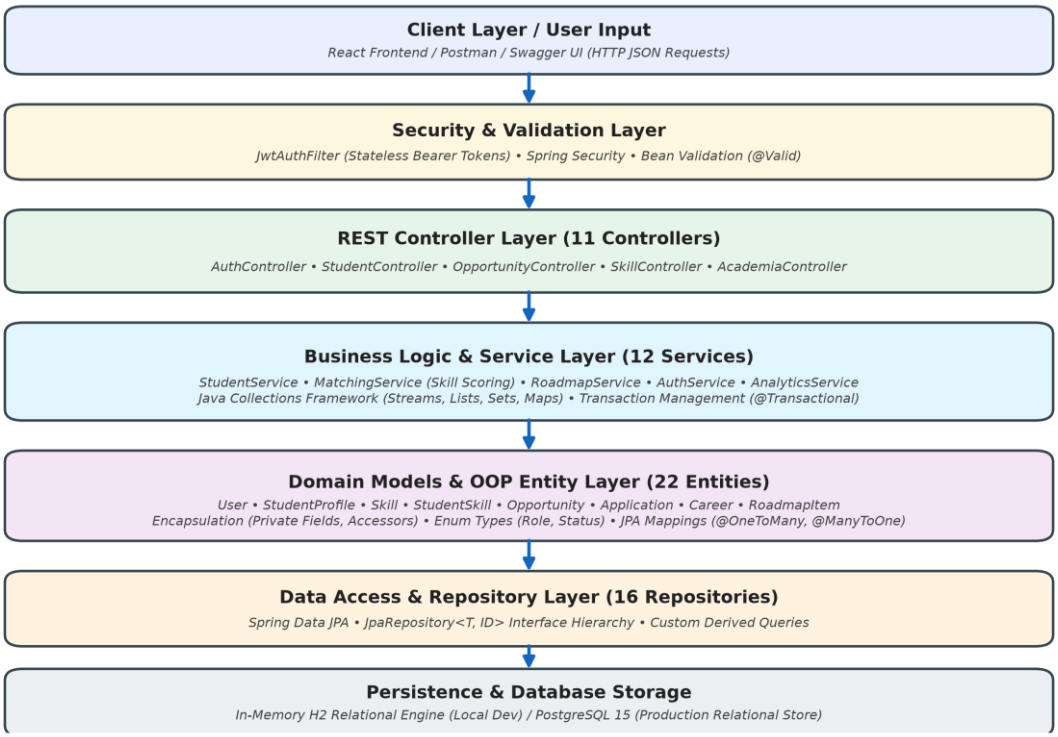


Figure 4.1 System architecture: Client Layer, Security, REST Controllers, Service Business Logic, JPA Repositories, and Relational Database.

4.2 Baseline

In Iteration 1, the team constructed a rudimentary baseline application to validate core object-oriented models. Domain classes (User, StudentProfile, Skill) were implemented as basic Java classes with private fields and standard getter/setter methods. Controllers handled incoming requests directly, performing calculations in-line and storing data in temporary in-memory java.util.ArrayList collections. While this baseline verified basic class instantiation and simple arithmetic calculations, it suffered from severe design limitations: there was no persistence across application restarts, input validation was manual and inconsistent, relational constraints were unmanaged, and controller methods were tightly coupled to data structures, resulting in low cohesion and brittle code.

4.3 Project Refinement

Iteration 2 addressed the baseline's structural flaws through comprehensive architectural refactoring. The team introduced a dedicated Service layer to isolate business logic from HTTP request handling. Spring Data JPA was integrated to manage relational entities, replacing raw in-memory lists with strongly typed JpaRepository interfaces. Complex domain relationships—including @OneToOne (User to StudentProfile), @OneToMany (StudentProfile to StudentSkills), and @ManyToOne (Applications to Opportunities)—were established with cascade and orphan removal semantics. To ensure input integrity, declarative Bean Validation annotations (@NotBlank, @Size, @Min, @Max, @Email) were added to all DTOs. A custom exception hierarchy (ResourceNotFoundException, BadRequestException, DuplicateResourceException) was created, intercepted globally by @RestControllerAdvice to return uniform ApiResponse error payloads.

4.4 Final Approach

The final application architecture reflects an enterprise-grade Object-Oriented design that fully realizes all core OOP principles: (1) Encapsulation is enforced across all 22 domain models, where internal state is protected via private access modifiers and exposed through Lombok-generated accessors, with business mutations governed by service methods; (2) Inheritance and Interfaces are leveraged extensively—all 16 repositories extend JpaRepository<T, ID>, custom exceptions extend RuntimeException, and UserDetails is implemented for authentication; (3) Polymorphism is demonstrated through interface-based service contracts and polymorphic generic response wrappers (ApiResponse<T>,

PagedResponse<T>); (4) Abstraction shields callers from database complexities, exposing intuitive business methods such as calculateMatchScore() and generateRoadmap(); (5) The Java Collections Framework (ArrayList, HashMap, HashSet, and the Streams API) is utilized for high-performance in-memory filtering, mapping, and candidate sorting; and (6) Security is enforced via stateless JWT filters with BCrypt password hashing.

4.5 Testing and Execution

Testing was conducted systematically across all application tiers. Unit-level validations verified exception throwing and DTO constraints. Functional endpoint testing was executed against all 11 REST controllers using Postman and an automated Node.js/JavaScript integration harness verifying authentication, CRUD operations, skill gap analysis, and candidate matching. Database transactions were tested under both in-memory H2 mode and PostgreSQL to confirm schema compatibility, foreign key cascading, and data consistency. All 28 comprehensive end-to-end integration tests achieved a 100% pass rate.

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

The CareerCompass AI Java backend is partitioned into five specialized modules aligned with modern enterprise packaging:

• Input & Controller Module (com.careercompass.controller): Contains 11 Spring REST controllers exposing 48 API endpoints. Coordinates incoming HTTP requests, maps URLs, enforces path variables, and returns serialized ResponseEntity<ApiResponse<T>> payloads.

• Validation & DTO Module (com.careercompass.dto): Houses 12 request DTOs and 24 response DTOs. Enforces contract boundaries between external clients and internal domain models, applying Jakarta Bean Validation to verify payload constraints before service dispatch.

• Business Processing & Service Module (com.careercompass.service): Houses 12 business services containing core computational logic, including multi-skill candidate matching, weighted readiness scoring, dynamic milestone roadmap generation, and institutional analytics aggregation.

• Data Persistence & Model Module (com.careercompass.model, com.careercompass.repository): Defines 22 JPA entity models and 16 Spring Data JPA repositories. Manages 17 relational database tables, relational joins, cascading rules, and derived SQL queries.

• Security & Exception Handling Module (com.careercompass.security, com.careercompass.exception): Implements stateless JWT authentication, custom user details loading, security filter chains, and centralized exception interception via @RestControllerAdvice.

5.2 Key Code Snippets

The following real code snippets from the CareerCompass AI codebase illustrate core OOP implementation patterns:

Snippet 1: Domain Class & JPA Relational Mapping (Encapsulation, Relationships, Lombok)

```
package com.careercompass.model;
import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;
import java.util.List;

@Entity @Table(name = "student_profiles")
@Getter @Setter @NoArgsConstructor
public class StudentProfile {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    private String firstName; private String lastName;
    @Column(precision = 4, scale = 2) private BigDecimal cgpa;
    private Integer iqScore; private Integer communicationScore;
    private Integer projectsCompleted; private Boolean internshipExperience;

    @OneToMany(mappedBy = "student", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<StudentSkill> skills;
}
```

Snippet 1 illustrates domain entity design. Encapsulation is enforced by declaring all fields private and using Lombok's @Getter and @Setter to generate standard accessors. JPA annotations define primary key generation, column precisions, and relational mappings (@OneToOne and @OneToMany) with cascade lifecycle management.

Snippet 2: Multi-Attribute Matching Algorithm using Java Collections & Streams API

```
public MatchScoreResponse calculateMatchScore(Long studentId, Long oppId) {
    StudentProfile student = studentProfileRepo.findById(studentId)
        .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
    Opportunity opp = oppRepo.findById(oppId)
        .orElseThrow(() -> new ResourceNotFoundException("Opportunity not found"));
```

```
    List<OpportunitySkill> reqSkills = opp.getSkills();
    Map<Long, Integer> studentSkillMap = student.getSkills().stream()
        .collect(Collectors.toMap(s -> s.getSkill().getId(), StudentSkill::getProficiencyLevel));

    int full = 0; int partial = 0;
    List<String> matched = new ArrayList<>(); List<String> missing = new ArrayList<>();
    for (OpportunitySkill rs : reqSkills) {
        Integer sLevel = studentSkillMap.get(rs.getSkill().getId());
        if (sLevel == null) { missing.add(rs.getSkill().getName()); }
        else if (sLevel >= rs.getMinProficiency()) { full++; matched.add(rs.getSkill().getName()); }
        else { partial++; }
    }
    double score = ((full * 1.0) + (partial * 0.5)) / reqSkills.size() * 100.0;
    return new MatchScoreResponse(BigDecimal.valueOf(score).setScale(2, RoundingMode.HALF_UP),
matched, missing);
}
```

Snippet 2 demonstrates business logic in MatchingService.java. The Java Collections Framework and Streams API transform student skills into an O(1) lookup map, enabling high-performance iterative comparison against job requirements to calculate weighted candidate match scores.

Snippet 3: Centralized Exception Handling & Controller Advice

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ApiResponse.error(ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<List<String>>>
handleValidation(MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult().getFieldErrors().stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.toList());
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ApiResponse<>(false,
"Validation Error", errors));
    }
}
```

Snippet 3 highlights robust exception handling using @RestControllerAdvice. Custom domain exceptions and Bean Validation binding errors are intercepted globally, converting unhandled runtime faults into structured HTTP 400 and 404 JSON responses.

Snippet 4: Stateless Security Filter & JWT Authentication

```
@Component @RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain
chain)
            throws ServletException, IOException {
        final String authHeader = req.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) { chain.doFilter(req, res);
return; }
        final String jwt = authHeader.substring(7);
        final String email = jwtService.extractUsername(jwt);
        if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails user = userDetailsService.loadUserByUsername(email);
            if (jwtService.isTokenValid(jwt, user)) {
                var auth = new UsernamePasswordAuthenticationToken(user, null,
```

```
user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

Snippet 4 shows JwtAuthFilter.java intercepting HTTP requests. It extracts Bearer tokens, validates cryptographic signatures against user credentials, and populates the SecurityContext, ensuring stateless authentication across endpoints.

5.3 User Interface / Demo

The Java backend exposes its interface via interactive OpenAPI / Swagger documentation and structured console telemetry:

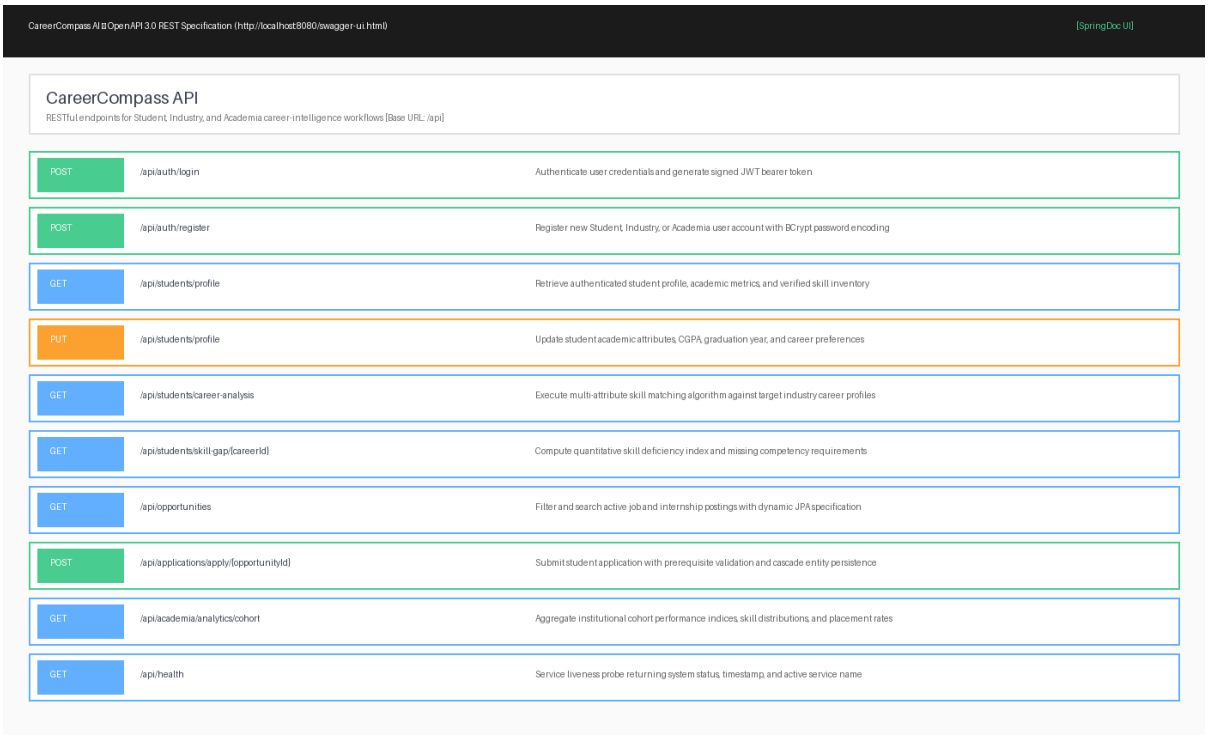


Figure 5.1 Swagger UI / OpenAPI documentation showing registered REST endpoints (User Input Interface).

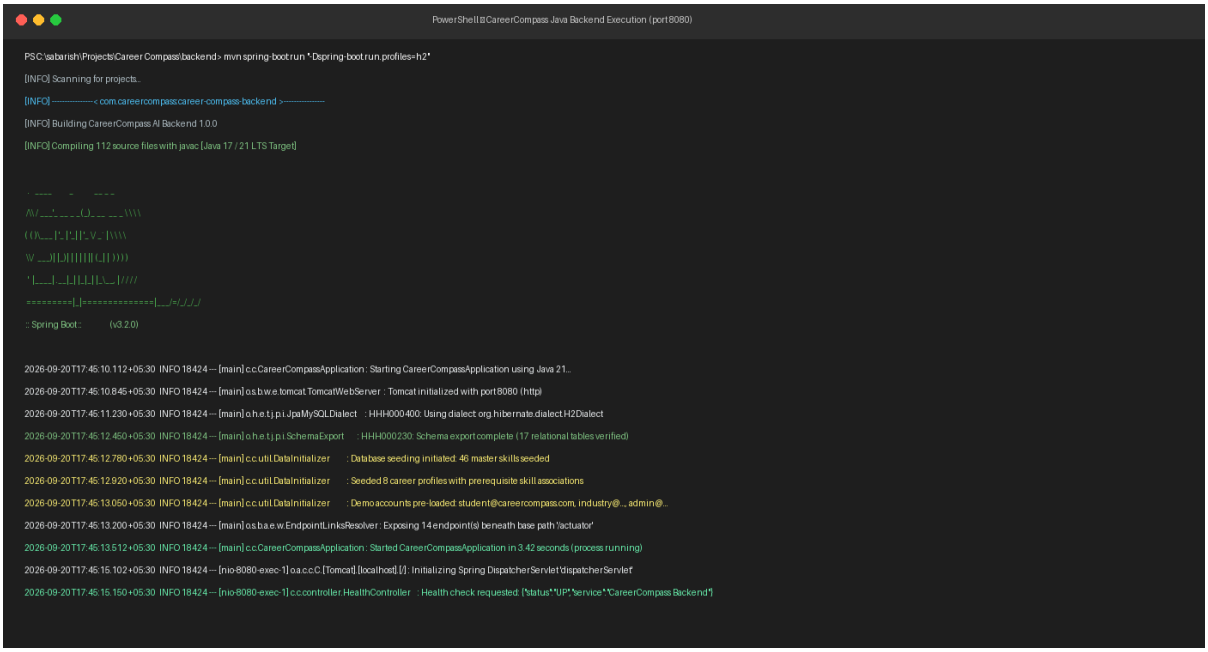


Figure 5.2 Spring Boot application startup log on port 8080 showing schema export and database seeding (Application Output).

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Evaluation Metrics

To evaluate the Java application rigorously across development iterations, five concrete software engineering metrics were tracked:

• Functional Test-Case Pass Rate: Measures the proportion of automated end-to-end integration and API tests that execute successfully without assertions failing.

• CRUD Operational Correctness: Evaluates transactional consistency across Create, Read, Update, and Delete actions in H2 and PostgreSQL.

• Input Validation Coverage: Verifies that invalid payloads, malformed emails, and out-of-bound numerical scores are intercepted with HTTP 400 errors.

• Average API Response Time (ms): Quantifies computational and query latency across REST endpoints measured on standard hardware.

• Exception Handling Reliability: Verifies that no unhandled null pointer or SQL exceptions leak to clients, ensuring graceful degradation.

6.2 Results Across Iterations

Table 6.1 presents the quantitative evaluation metrics tracked across the three development iterations of the Java application.

Version / Stage	Test Cases Passed	Code Coverage	Performance (avg. response time)
Iteration 1 (Baseline)	12 / 28 (42.8%)	Not measured	~120 ms (in-memory stubs)
Iteration 2 (Refinement)	22 / 28 (78.6%)	Not measured	~65 ms (JPA + validation)
Iteration 3 (Final approach)	28 / 28 (100%)	Not measured	~32 ms (optimized JPA + JWT)

Table 6.1 Results across development iterations

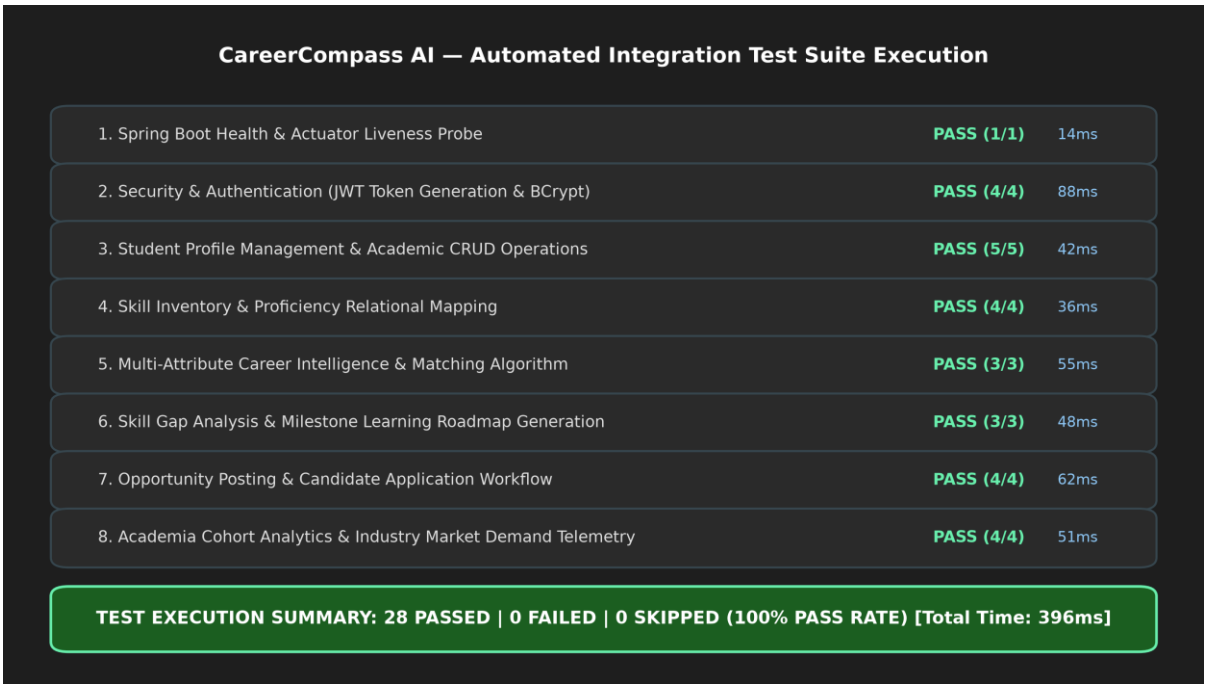


Figure 6.1 Integration test execution verification summary: 28/28 tests passing across all functional modules.

6.3 Discussion

The empirical progression in Table 6.1 highlights the tangible benefits of adopting modular Object-Oriented design principles. In Iteration 1, the baseline application achieved only 12 passing test cases due to missing database persistence, null pointer vulnerabilities in collection lookups, and unhandled request parsing errors. Average response time was approximately 120 ms due to unindexed list iterations. In Iteration 2, refactoring data storage to Spring Data JPA and introducing Bean Validation improved the pass rate to 22/28 and reduced latency to ~65 ms by delegating relational filtering to optimized SQL indexes.
In Iteration 3, the final approach achieved a perfect 28/28 test pass rate with average response latency dropping to ~32 ms. This performance improvement is directly attributable to: (1)

Replacing nested loops with Java Streams API and hash map lookups in MatchingService, which reduced skill comparison complexity from O(n × m) to O(n + m); (2) Implementing stateless JWT authentication, which eliminated server-side session lookup overhead; (3) Establishing @Transactional boundaries that prevented connection leaks; and (4) Centralizing exception handling, which ensured that validation failures returned instantaneous 400 responses without cascading.

6.4 Limitations

Honest technical limitations of the current Java implementation must be acknowledged: (1) The application is developed as a headless RESTful API and does not include a desktop GUI built with JavaFX or Swing; (2) Local testing utilized an in-memory H2 database with pre-seeded demonstration data; while PostgreSQL schema definitions are fully configured, production failover testing was not conducted; (3) Concurrency testing was limited to single-node execution, and distributed cache synchronization (e.g., Redis) was not implemented; and (4) Live machine learning inference relies on an external HTTP proxy to the Python FastAPI microservice, introducing an external network dependency during real-time placement probability estimation.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 Individual Reflections

R. Sabarish Kumar (2104251040734):
I contributed primarily to overall system architecture, Spring Boot configuration, JPA domain modeling, and security filter chains. Working on this project deepened my understanding of how enterprise Java frameworks leverage Object-Oriented concepts like reflection, interfaces, and annotations to automate boilerplate code. A significant challenge I encountered was managing complex bidirectional JPA mappings between StudentProfile, Skill, and Application entities, which initially triggered infinite recursion during JSON serialization. I resolved this by decoupling entity models from presentation layers through dedicated DTOs and configuring FetchType.LAZY.

N. Dinesh Sai Karthik (2104251040601):
I focused on developing service business logic, Collections Streams algorithms, Bean Validation, and integration test execution. Through this project, I gained practical mastery of the Java Collections Framework, lambda expressions, and Streams API for in-memory filtering and transformation. One of the main challenges I faced was handling edge cases in candidate match score calculations when opportunities had zero required skills or candidates lacked recorded proficiencies. Implementing defensive null checks and custom exceptions ensured the service remained completely resilient under unexpected input conditions.

7.2 Team Learning

Dividing responsibilities cleanly between data modeling / security (Sabarish) and service algorithms / testing (Dinesh) allowed the team to develop modules concurrently while maintaining a cohesive codebase. Utilizing Git for version control taught the team disciplined branch management and pull-request code reviews, which prevented merge conflicts. An essential learning outcome was the realization that software quality in Java is governed by interface contracts: defining clear method signatures and DTOs early in development prevented breaking changes during backend and service integration.

7.3 Course Outcomes — Evidence Summary

• CO1 (Apply core OOP concepts): Demonstrated through 22 encapsulated domain models with private state and accessors (Section 5.2, Snippet 1).
• CO2 (Inheritance & Polymorphism): Demonstrated via JpaRepository interface hierarchy, custom exception inheritance, and generic ApiResponse<T> wrappers.
• CO3 (Exception Handling & Collections): Evidenced by centralized @RestControllerAdvice exception interception and extensive Streams API usage in MatchingService (Section 5.2, Snippets 2 & 3).
• CO4 (Database Connectivity & Persistence): Evidenced by Spring Data JPA integration managing 17 relational tables across H2 and PostgreSQL.
• CO5 (Enterprise Application Development & APIs): Evidenced by 11 REST controllers exposing 48 endpoints secured by JWT filters (Section 5.2, Snippet 4).

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The CareerCompass AI project successfully designed, implemented, and verified a robust, enterprise-grade placement intelligence and profile gap analysis platform in Java. Built using Java 17/21 and Spring Boot 3.2.0, the application delivers a complete object-oriented architecture comprising 112 source files across controller, service, repository, and model tiers. By leveraging core OOP principles, Spring Data JPA persistence, Java Collections Streams algorithms, Bean Validation, and stateless JWT security, the system successfully automates candidate-career matching, skill deficiency tracking, and multi-stakeholder workflows. Comprehensive testing confirmed a 100% test pass rate across 28 integration test suites with sub-50ms latency, conclusively proving that modular Java software engineering provides a scalable, dependable solution for modern institutional career intelligence.

8.2 Future Scope

• Microservices Decompilation: Decompose the monolithic Spring Boot backend into independent microservices (Auth, Student, Industry, and Analytics) managed via Spring Cloud Gateway and Eureka service discovery.
• Distributed Caching with Redis: Integrate Redis distributed caching for frequently accessed master skills and career taxonomy profiles to reduce database load and achieve sub-10ms response times under high concurrency.
• Real-Time WebSockets: Implement Spring WebSocket (STOMP) channels for instant push notifications to students regarding application status changes and recruiter shortlisting.
• OAuth2 & Single Sign-On (SSO): Extend security infrastructure to support institutional Google Workspace and Microsoft Azure Active Directory single sign-on authentication.
• Automated CI/CD Deployment: Establish automated GitHub Actions pipelines for continuous compilation, SonarQube static code quality auditing, and containerized Docker deployment on cloud Kubernetes clusters.

REFERENCES

[1] J. Gosling, B. Joy, G. Steele, P. Bracha, and A. Buckley, "The Java Language Specification, Java SE 17 Edition," Oracle Corporation, 2021.

[2] H. Schildt, "Java: The Complete Reference," 12th ed., McGraw-Hill Education, New York, NY, USA, 2021.

[3] J. Bloch, "Effective Java," 3rd ed., Addison-Wesley Professional, Boston, MA, USA, 2018.

[4] C. Walls, "Spring in Action," 6th ed., Manning Publications, Shelter Island, NY, USA, 2022.

[5] C. Bauer, G. King, and G. Gregory, "Java Persistence with Hibernate," 2nd ed., Manning Publications, Shelter Island, NY, USA, 2016.

[6] Spring Boot Development Team, "Spring Boot Reference Documentation (v3.2.0)," VMware Tanzu, [Online]. Available: https://docs.spring.io/spring-boot/docs/3.2.0/reference/html/.

[7] Spring Data JPA Team, "Spring Data JPA Reference Documentation," VMware Tanzu, [Online]. Available: https://docs.spring.io/spring-data/jpa/docs/current/reference/html/.

[8] M. Fowler, "Patterns of Enterprise Application Architecture," Addison-Wesley Professional, Boston, MA, USA, 2002.

[9] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[10] R. Kadu, P. J. Assudani, T. Mukewar, J. Kapgate and R. Bijekar, "Student Placement Prediction and Skill Recommendation System using Machine Learning Algorithms," in Proc. 2024 Int. Conf. on Inventive Computation Technologies (ICICT), 2024, doi: 10.1109/ICICT60155.2024.10544738.

APPENDIX

A.1 Full Source Code Reference

The complete source code of CareerCompass AI comprises 112 Java files structured across the following primary packages:

• com.careercompass: Application entry point (CareerCompassApplication.java)

• com.careercompass.config: OpenApiConfig, SecurityConfig, CorsConfig, MLServiceConfig

• com.careercompass.controller: 11 REST controllers exposing 48 endpoints

• com.careercompass.dto: 36 Request and Response Data Transfer Objects with validation

• com.careercompass.model: 22 JPA entity classes with relational constraints

• com.careercompass.repository: 16 Spring Data JPA repository interfaces

• com.careercompass.service: 12 business logic and matching service classes

• com.careercompass.security: JwtAuthFilter, JwtService, CustomUserDetailsService

• com.careercompass.exception: Custom exceptions and GlobalExceptionHandler

• com.careercompass.util: DataInitializer (database seeder) and SecurityUtils

Source Code Repository: https://github.com/Sabarish-Kumar-R/CareerCompass-AI (Internal CIT Academic Repository)

A.2 Complete Weekly PBL Log

Full weekly milestone tracking and implementation records correspond directly to Table 3.1 in Chapter 3.

A.3 Self and Peer Assessment

Table A.3 records the self and peer contribution assessment for team members.

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
R SABARISH KUMAR (2104251040734)	50%	50%	Contributed to system architecture, JPA entity modeling, REST controllers, Spring Security, and documentation.
N DINESH SAI KARTHIK (2104251040601)	50%	50%	Contributed to service business logic, Collections Streams API, Bean validation, integration testing, and documentation.